

Sobre el uso de IAG:

Nivel de uso de IAG	Descriptor
3. Colaboración con la IAG	En esta actividad se puede utilizar cualquier herramienta / modelo basado en IAG para refinar, editar y mejorar cualquier requerimiento de la fase de codificación. Dentro de los entregables se debe incluir un documento de reflexión sobre su uso.

Esta tarea integradora presenta una actividad en la cual se requiere aplicar todos los conocimientos adquiridos a lo largo del curso, por lo tanto, es un instrumento para verificar el cumplimiento de los objetivos de aprendizaje que han sido planteados en el programa del curso. A continuación, encontrará un enunciado que narra de forma detallada la situación problema que se espera usted solucione.

Enunciado

La empresa DreamTeam S.A.S., dedicada a la organización de actividades de integración para instituciones educativas y organizaciones, solicita el desarrollo del software SorteoSecreto para gestionar de forma eficiente, confiable y segura la dinámica de Amigo Secreto para grupos de distintos tamaños. Actualmente, estos procesos se realizan de manera manual o con herramientas informales, lo que ha generado errores en los sorteos, incumplimiento de reglas y problemas de confidencialidad.

El sistema deberá manejar usuarios que pueden asumir distintos roles según el sorteo. Un mismo usuario puede crear un sorteo y, para ese sorteo, actuar como organizador (con permisos de administración). En otros sorteos, ese mismo usuario podrá participar solo como participante. Por tanto, los permisos se definen por sorteo: el organizador administra la configuración del sorteo (creación, edición, importación y gestión de participantes, definición de restricciones, ejecución/anulación y clonación de sorteo), mientras que el participante accede únicamente a la información habilitada para él dentro de ese sorteo (por ejemplo, ver su amigo secreto y gestionar su lista de deseos), considerando que estas opciones pueden variar según la configuración del sorteo.

El sistema debe permitir que un usuario con rol de organizador cree un sorteo definiendo su nombre, descripción, presupuesto sugerido y fecha de realización. El organizador podrá adicionar participantes ya sea de forma individual o importarlos desde un archivo (funcionalidad asignada exclusivamente a estudiantes de Ingeniería de Sistemas), así como editar o eliminar registros antes del cierre del sorteo, garantizando validaciones básicas como la no duplicidad de participantes. Cada participante se identifica por su correo electrónico, el cual debe ser único dentro del sistema; no se permitirá registrar (ni importar) dos participantes con el mismo correo. A continuación, un ejemplo del archivo:

email	nombre
ana.perez@correo.com	Ana Pérez
carlos.rojas@empresa.edu	Carlos Rojas

El sistema debe ejecutar un sorteo automático que asigne a cada participante exactamente una persona a quien regalar, asegurando que nadie se asigne a sí mismo, que no existan repeticiones y que se respeten restricciones de asignación, definidas por el organizador, tales como grupos o pares de

Commented [JM1]: Lo dejo resaltado para no olvidar quitar este texto en el definitivo

Commented [JC2R1]: Se puede dejar, no hay problema

participantes que no pueden sacarse entre sí. En caso de que las restricciones hagan inviable el sorteo, el sistema deberá detectarlo y notificar al organizador sin publicar resultados.

Cada sorteo deberá manejar estados claros (por ejemplo: creado, cerrado, sorteado, anulado). El organizador podrá anular un sorteo, dejándolo sin efecto y bloqueando el acceso a las asignaciones, y también clonar un sorteo existente para crear una nueva edición, ajustando datos específicos como nombre, presupuesto y fecha, y adicionando nuevos integrantes sin perder la configuración general.

Las asignaciones deben mantenerse confidenciales, de modo que cada participante solo pueda conocer a quién le corresponde, ya sea mediante el sistema o por notificación. El organizador solo podrá visualizar información de estado general, de si el participante ya consultó su amigo, no los emparejamientos individuales.

Cada participante podrá gestionar una lista de deseos asociada a un sorteo, en la cual podrá registrar elementos sugeridos para el regalo. Los elementos de la lista de deseos deberán soportar distintos tipos con atributos específicos, por ejemplo:

- **Tecnología:** nombre, marca, modelo o referencia, y otras especificaciones relevantes (color, tamaño, etc.).
- **Prenda de vestir:** nombre, categoría (camiseta, pantalón, chaqueta, etc.), marca y talla.
- **Libro:** título, autor, editorial, y género.
- **Experiencia/Actividad:** nombre, tipo de experiencia (cine, cena, curso, etc.), ciudad o ubicación, y otras restricciones (como horarios o preferencias).
- **Accesorio:** nombre, tipo (reloj, bolso, billetera, etc.), marca y material.

El software debe estar diseñado de tal manera que en el futuro se puedan crear nuevos tipos de elementos.

El sistema debe permitir que un participante exporte a un archivo de texto legible (para estudiantes de Ingeniería de Sistemas) o muestre por consola (para estudiantes de Diseño de Medios Interactivos e Ingeniería en Energía Inteligente) la información de la lista de deseos de su amigo secreto, de manera que pueda consultarla cuando lo necesite. La exportación o visualización por consola deberá incluir el nombre del amigo secreto y los datos relevantes de cada elemento de la lista de deseos (según su tipo). La generación del archivo o la visualización por consola solo podrá realizarse una vez el sorteo haya sido ejecutado y el participante tenga asignado su amigo secreto. El participante únicamente podrá exportar o visualizar la lista de deseos de la persona que le fue asignada, sin acceso a la información de otros participantes.

Una vez finalizado el sorteo y validado que la fecha actual sea posterior a la fecha definida del sorteo, el sistema deberá permitir la visualización de un reporte consolidado que muestre quién sacó a quién, presentado en un formato tipo matriz (participantes en filas y amigo secreto en columnas, o equivalente). Este reporte será accesible exclusivamente para el organizador y solo cuando el sorteo haya concluido, con el fin de preservar la confidencialidad durante la ejecución y la entrega de regalos.

Durante el desarrollo del proyecto, los equipos deberán utilizar un sistema de control de versiones basado en Git, empleando repositorios alojados en GitHub. De acuerdo con las políticas de la empresa, cada

desarrollador debe mantener su repositorio actualizado y estructurado de forma clara, con ramas dedicadas para desarrollo, integración y versión final. Los mensajes de commit deben seguir una notación consistente y significativa, recomendándose un formato tipo "tipo: descripción breve" (por ejemplo, feat: agregar lógica de revelación automática). Se exige el uso del lenguaje de programación Java, en su versión 17 o superior, siguiendo estándares de codificación bien establecidos. Todo el código fuente debe estar debidamente comentado, utilizando documentación Javadoc para las clases y métodos principales. Además, se espera que los equipos estructuren su proyecto siguiendo buenas prácticas de organización de paquetes, división de intereses y responsabilidades y modularidad del código, y que desarrollen pruebas unitarias que permitan verificar y garantizar la funcionalidad implementada cumpliendo con los plazos definidos.

Primera fase de desarrollo

En la primera fase de desarrollo la solución debe estar escrita en una sola clase que contiene tanto el método main como métodos auxiliares definidos para separar funcionalidades como entrada de datos, validaciones, cálculos y visualización de resultados. A continuación, se detallan los requerimientos funcionales:

1. **Registro de información de un sorteo:** El sistema debe permitir al usuario registrar la información de UN solo sorteo. El sorteo está identificado por un nombre (cadena de texto), cuenta con descripción (cadena de texto), presupuesto sugerido por regalo (número decimal), y fecha del evento (clase para manejar fechas: ver Anexo). El sorteo cuenta adicionalmente con un estado (creado o sorteado).
2. **Registro de información de un participante:** El sistema debe permitir registrar la información de los participantes de forma manual. Para ello se debe solicitar el número de participantes y registrar la información de tantos participantes como corresponda. Para cada participante se requiere el nombre (cadena de texto).
3. **Validar información de un participante:** El sistema debe validar que no existan dos participantes con el mismo nombre dentro del sorteo.
4. **Consultar participantes:** El sistema debe permitir mostrar por consola la lista completa de participantes registrados, presentando un consecutivo y el nombre.
5. **Generar sorteo:** El sistema debe, a solicitud del usuario, realizar un sorteo automático que asigne a cada participante exactamente un amigo secreto (nombre) de manera aleatoria, asegurando que nadie se asigne a sí mismo y que no existan repeticiones (clase para manejar aleatoriedad: ver Anexo).
6. **Resumen del sorteo:** El sistema debe, a solicitud del usuario, presentar por consola los datos del sorteo. Si el sorteo ya fue realizado se debe consultar el listado de participantes y su amigo secreto. En caso contrario, se mostrará un mensaje de error.

Tenga en cuenta que el sistema debe ofrecer un **menú de opciones en consola** que permita al usuario acceder a las funcionalidades del sistema. Este menú se debe desplegar hasta que el usuario decida salir del sistema.

Entregables de la primera fase de desarrollo

Fecha de entrega: Hasta el sábado 07 de marzo de 2026 a las 21:59.

Lleve a cabo las siguientes actividades de cada una de las etapas de desarrollo de software:

1. **Análisis del problema.** Identificación y análisis del problema en el encabezado de la clase. Incluya: descripción del problema general, y para cada requerimiento: descripción del requerimiento, entradas (con su tipo de dato), salidas (con su tipo de dato) y ejemplo de ejecución.
2. **Implementación EN INGLÉS del código Java para la Primera Fase de Desarrollo.** Incluya, si lo requiere comentarios descriptivos sobre el código java escrito por usted.
3. **Documentación.** Se deben incluir los contratos de los métodos implementados: descripción, parámetros y retorno.
4. Usar **GitHub como repositorio de código fuente y documentación.** Recuerde que se debe evidenciar su avance a través de los días en el desarrollo de su tarea (al menos 3 commits significativos).

Rúbrica de evaluación

En el siguiente enlace puede encontrar la rúbrica con la cual será evaluado en esta entrega: [enlace a rúbrica](#).

Organización de los repositorios

Tenga en cuenta que su repositorio GitHub debe presentar una estructura base como, por ejemplo:

SorteoSecreto/

La carpeta raíz del proyecto contendrá su clase .java y el archivo .class generado luego de la compilación.

Anexo - LocalDate

Se recomienda utilizar la clase LocalDate (ubicada en java.time) para realizar el manejo de fechas en Java. Lo invitamos a revisar la documentación: <https://docs.oracle.com/javase/8/docs/api/java/time/LocalDate.html>

Para usar los métodos de esta clase, primero debe importarlo desde el paquete correspondiente:

```
import java.time.LocalDate;
```

Para obtener una instancia de LocalDate se pueden emplear diferentes métodos:

```
LocalDate date = LocalDate.of(year, month, dayOfMonth); //Donde year, month y dayOfMonth son enteros.
```

```
LocalDate date = LocalDate.now(); //Para obtener la fecha del sistema en el momento exacto de ejecución.
```

Ejemplo:

```
import java.time.LocalDate;

public class Main {

    public static void main (String[] args){
        int year = 2020;
        int month = 10;
        int dayOfMonth = 5;

        LocalDate date = LocalDate.of(year, month, dayOfMonth);
        System.out.println("La fecha definida con parámetros es: "+date);

        LocalDate now = LocalDate.now();
        System.out.println("La fecha del sistema es: "+now);
    }
}
```

La salida obtenida por consola (al momento de construcción de este enunciado) es:

La fecha definida con parámetros es: 2020-10-05

La fecha del sistema es: 2026-02-08

Anexo – Random

Se recomienda utilizar la clase Random (ubicada en java.util) para generar valores pseudoaleatorios en Java. Esta clase es ampliamente utilizada en simulaciones, juegos, pruebas y en cualquier escenario donde se requiera variabilidad controlada. Lo invitamos a revisar la documentación oficial:

<https://docs.oracle.com/javase/8/docs/api/java/util/Random.html>

Para usar los métodos de esta clase, primero debe importarlo desde el paquete correspondiente:

```
import java.util.Random;
```

Para generar valores aleatorios, primero se debe crear una instancia de la clase Random:

```
Random random = new Random(); //Generador con semilla automática basada en el tiempo del sistema.
```

Algunos métodos comunes de la clase Random son:

```
random.nextInt(); //Genera un entero aleatorio (puede ser positivo o negativo).
random.nextInt(bound); //Genera un entero entre 0 (incluido) y bound (excluido).
random.nextDouble(); //Genera un número decimal entre 0.0 (incluido) y 1.0 (excluido).
random.nextBoolean(); //Genera un valor booleano (true o false).
```

Ejemplo:

```
import java.util.Random;

public class Main {

    public static void main (String[] args){
        Random random = new Random();

        int numEntero = random.nextInt(10); //Entre 0 y 9

        System.out.println("Entero aleatorio entre 0 y 9: " + numEntero);

        double numDecimal = random.nextDouble();

        System.out.println("Decimal aleatorio entre 0.0 y 1.0: " + numDecimal);
        boolean valorBooleano = random.nextBoolean();

        System.out.println("Valor booleano aleatorio: " + valorBooleano);
    }
}
```

La salida obtenida por consola puede variar en cada ejecución, por ejemplo:

```
Entero aleatorio entre 0 y 9: 7
Decimal aleatorio entre 0.0 y 1.0: 0.5348291738291
Valor booleano aleatorio: true
```

Nota: Los valores generados por **Random** son pseudoaleatorios, es decir, siguen un algoritmo determinístico.